UNIVERSIDAD
DE GRANADA

Universidad de Granada

ETSIIT

Memoria Práctica 1

Autores:

Javier Miñán Molina
Germán Delgado Fernández
Marcos García Álvarez

Desarrollo de Software

22 de marzo de 2026

Índice

1. Ejercicio1: Simulación de Partidas (Competitiva vs. Casual) 2
1.1. Arquitectura y Patrones de Diseño Aplicados . . . 2
1.2. Diagrama de clases UML . . . 2
1.3. Flujo de Ejecución y Configuración Dinámica . . . 3
1.4. Simulación Concurrente (Multihilo) . . . 4
1.5. Pruebas de Ejecución . . . 4

2. Ejercicio 2: Procesamiento de Lenguaje Natural con el Patrón Decorator 6
2.1. Objetivo de la práctica . . . 6
2.2. Arquitectura y Patrones de Diseño Aplicados . . . 6
2.3. Implementación y Flujo de Datos . . . 6
2.4. Modo Interactivo y Flexibilidad del Patrón (Extra) . . . 7
2.5. Diagrama de clases UML . . . 7
2.6. Pruebas de Ejecución . . . 8

3. Ejercicio 3: Extracción de Datos Estadísticos (Web Scraping) 9
3.1. Arquitectura y Patrones de Diseño Aplicados . . . 9
3.2. Flujo de Ejecución y Extracción de Datos . . . 9
3.3. Implementación de las Estrategias (BeautifulSoup vs. Selenium) . . . 10
3.4. Almacenamiento . . . 11
3.5. Diagrama de clases UML . . . 12
3.6. Pruebas de Ejecución . . . 12

4. Ejercicio 4: Patrón Intercepting Filter 14
4.1. Introducción . . . 14
4.2. Descripción de la solución . . . 14
4.3. Uso de herencia vs módulos . . . 14
4.4. Diagrama UML . . . 14
4.5. Ejemplos de ejecución . . . 15
4.5.1. Caso válido . . . 15
4.5.2. Caso inválido: Formato incorrecto . . . 15
4.5.3. Caso inválido: Dominio incorrecto . . . 15
4.5.4. Caso inválido: Longitud de contraseña incorrecta . . . 15
4.5.5. Caso inválido: Contraseña con al menos un número . . . 15
4.5.6. Caso inválido: Contraseña con al menos un carácter especial . . . 16
4.6. Conclusiones . . . 16

1. Ejercicio1: Simulación de Partidas (Competitiva vs. Casual)

1.1. Arquitectura y Patrones de Diseño Aplicados

Para este ejercicio se ha optado por utilizar el **Patrón de Diseño Abstract Factory** (Fábrica Abstracta). Este patrón es ideal para este escenario porque nos permite crear familias de objetos relacionados (Partidas y Jugadores) sin especificar sus clases concretas en el código cliente (`Main`).

- **Interfaz/Clase Abstracta** `FactoriaPartidaYJugador`: Define el contrato para crear los elementos del juego.

```java
public interface FactoriaPartidaYJugador {
    Jugador crearJugador(int id);
    Partida crearPartida(int numero_jugadores);
}
```

- **Fábricas Concretas** (`FactoriaCasual` **y** `FactoriaCompetitiva`): Implementan la interfaz anterior. Su responsabilidad es garantizar que no se mezclen elementos incompatibles; es decir, la fábrica casual siempre instanciará una partida con reglas casuales y jugadores configurados para ese modo, mientras que la competitiva hará lo propio con su ecosistema.

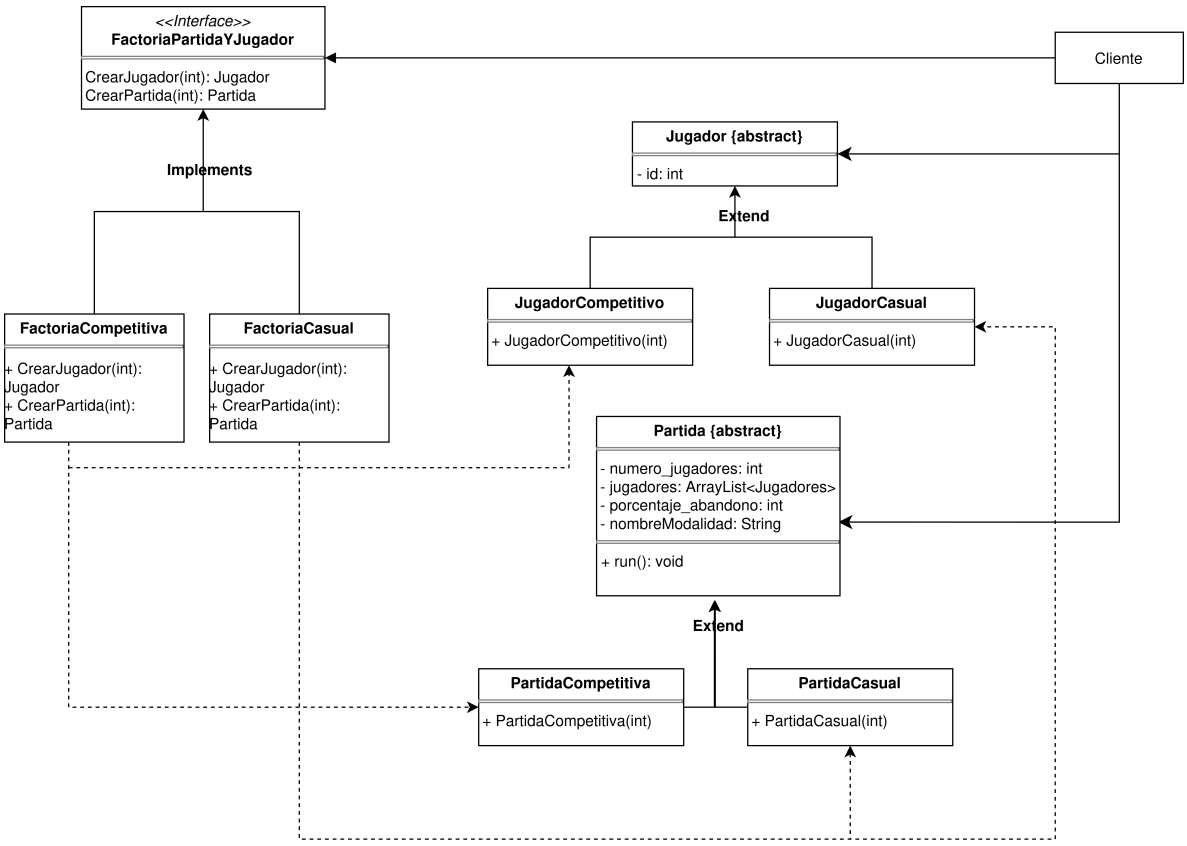
```java
public class FactoriaCompetitiva implements FactoriaPartidaYJugador {
    @Override
    public Jugador crearJugador(int id) {
        return new JugadorCompetitivo(id);
    }

    @Override
    public Partida crearPartida(int numero_jugadores) {
        return new PartidaCompetitiva(numero_jugadores);
    }
}
```

- **Desacoplamiento**: Gracias a este diseño, el método `main` no necesita conocer los detalles internos de cómo se construye un jugador competitivo o qué parámetros de abandono tiene una partida casual. Si en el futuro se desea añadir un nuevo modo (por ejemplo, `FactoriaTorneo`), el código cliente apenas sufriría modificaciones.

1.2. Diagrama de clases UML

Para ilustrar la estructura estática del sistema y las relaciones entre las interfaces y sus implementaciones, se presenta a continuación el diagrama de clases UML. En él se puede observar claramente cómo el patrón **Abstract Factory** aísla la creación de las familias de objetos (`Partida` y `Jugador`) del código cliente.



1.3. Flujo de Ejecución y Configuración Dinámica

El programa es interactivo y escalable:

- Se utiliza la clase `Scanner` para permitir al usuario definir dinámicamente el volumen de la simulación (número de jugadores).
- Mediante un bucle `for`, las fábricas instancian a los jugadores y los inyectan en sus respectivas partidas.
- Cada tipo de partida inicializa sus propias variables de estado, como el `porcentaje_abandono`, que lógicamente difiere en la partidaCasual y en PartidaCompetitiva (donde suele haber penalizaciones).

```java
import java.util.ArrayList;

public class PartidaCasual extends Partida {
    public PartidaCasual(int numero_jugadores) {
        jugadores = new ArrayList<>(numero_jugadores);
        this.numero_jugadores = numero_jugadores;
        porcentaje_abandono = 10;
        nombreModalidad = "CASUAL";
    }
}
```

1.4. Simulación Concurrente (Multihilo)

Para modelar el comportamiento real de los servidores de videojuegos, donde múltiples partidas ocurren simultáneamente, se ha implementado programación concurrente utilizando la API de hilos de Java:

- **Ejecución Paralela**: Las clase abstracta `Partida` implementa la interfaz `Runnable`, lo que permite pasar como argumento las clases `PartidaCasual` y PartidaCompetitiva a `new Thread()`. Al llamar a `hiloCasual.start()` y `hiloCompetitiva.start()`, ambas simulaciones arrancan en paralelo.

```java
@Override
public void run() {
    System.out.println("Comenzando partida " + nombreModalidad + "\n");

    int jugadoresAbandonan = numero_jugadores * porcentaje_abandono / 100;

    Random random = new Random();
    int tiempoAbandono = random.nextInt(40000) + 10000;

    try {
        Thread.sleep(tiempoAbandono);
    } catch (InterruptedException e) {
        System.out.println("La partida " + nombreModalidad + " fue interrumpida.");
    }

    System.out.println("Jugadores que abandonaron la partida " + nombreModalidad + ": " + jugad
    for (int i = 0; i < jugadoresAbandonan && !jugadores.isEmpty(); i++) {
        int jugadorAEliminar = random.nextInt(jugadores.size());
        System.out.println("Jugador " + jugadores.get(jugadorAEliminar).id + " ha abandonado la
        jugadores.remove(jugadorAEliminar);
    }
    System.out.println("\n");

    try {
        Thread.sleep(60000 - tiempoAbandono);
    } catch (InterruptedException e) {
        System.out.println("La partida " + nombreModalidad + " fue interrumpida.");
    }

    System.out.println("Finalizando partida " + nombreModalidad);
}
```

- **Sincronización (`join`)**: Para evitar que el hilo principal (`Main`) termine y cierre la aplicación antes de que finalicen las simulaciones, se utiliza el método `join()`. Esto obliga al programa a esperar a que ambas partidas concluyan su ejecución de forma segura, manejando adecuadamente la posible `InterruptedException`.

1.5. Pruebas de Ejecución

Para verificar el correcto funcionamiento de la simulación y de la concurrencia, se ha ejecutado el programa interactuando a través de la consola.

En la siguiente captura se puede observar cómo el sistema solicita dinámicamente el número de jugadores, instancia las fábricas correspondientes y lanza los hilos. Además, los mensajes de salida por consola demuestran que ambas simulaciones (`PartidaCasual` y `PartidaCompetitiva`) se ejecutan en paralelo de forma asíncrona, finalizando de forma segura gracias a la sincronización del método `join()`.

```
Picked up _JAVA_OPTIONS: -Dsun.java2d.uiScale=3
Introduce el número de jugadores para las partidas:
10
Partida Casual - Porcentaje de abandono: 10%
Partida Competitiva - Porcentaje de abandono: 20%
Número de jugadores en ambas partidas: 10

Comenzando partida CASUAL

Comenzando partida COMPETITIVA

Jugadores que abandonaron la partida COMPETITIVA: 2
Jugador 3 ha abandonado la partida COMPETITIVA
Jugador 8 ha abandonado la partida COMPETITIVA


Jugadores que abandonaron la partida CASUAL: 1
Jugador 4 ha abandonado la partida CASUAL


Finalizando partida CASUAL
Finalizando partida COMPETITIVA
```

2. Ejercicio 2: Procesamiento de Lenguaje Natural con el Patrón Decorator

2.1. Objetivo de la práctica

El objetivo es implementar un sistema de procesamiento de textos (LLM) que sea extensible. Partimos de una funcionalidad base (Resumen) y queremos añadirle çapas.ªdicionales como la Traducción y el Análisis de Sentimientos sin modificar el código de la clase original.

2.2. Arquitectura y Patrones de Diseño Aplicados

Se ha optado por utilizar el **Patrón de Diseño Decorator** (Estructural) porque nos permite añadir responsabilidades a un objeto de forma dinámica. En este ejercicio, la funcionalidad básica (el núcleo) es el resumen, y la decoración son los servicios extra de traducción y sentimiento.

Jerarquía de clases implementada:

- **Interfaz** `InterfaceLLM`: Una interfaz que define el contrato del sistema, estableciendo el método `generate_summary(texto)`. Ni el Componente Concreto ni los Decoradores pueden existir sin cumplir este contrato.

```python
from abc import ABC, abstractmethod

    # Define el contrato para todas las clases de procesamiento LLM.

class InterfaceLLM(ABC):
    @abstractmethod
    def generate_summary(self, text: str):
        # Este metodo recibe un texto y las clases que hereden implementaran la logica.
        pass
```

- **Componente Concreto (`BasicLLM`)**: Es la clase que hace la llamada real a la API de Hugging Face para resumir el texto original en inglés. Es el objeto que será envuelto por los decoradores.

- **Decorador Base (`LLMDecorator`)**: Clase que contiene una referencia al componente (de tipo `InterfaceLLM`) que está envolviendo. Su objetivo es servir de molde y delegar la ejecución en el objeto interno.

```python
class LLMDecorator(InterfaceLLM):

    def __init__(self, llm: InterfaceLLM):
        self.llm_envuelto = llm

    def generate_summary(self, text: str) -> str:
        # Por defecto, solo llama al método del objeto que tiene dentro
        return self.llm_envuelto.generate_summary(text)
```

- **Decoradores Concretos (`TranslationDecorator` y `SentimentDecorator`)**: Clases que heredan del decorador base y añaden su propia lógica (llamar a otros modelos de IA de Hugging Face) modificando el texto antes o después de que lo procese el objeto envuelto.

2.3. Implementación y Flujo de Datos

El programa lee la configuración de un archivo `config.json` (modelos, token y texto). La cadena completa usando todas las posibilidades (Prueba 4) sigue este flujo:

1. Se crea el `BasicLLM`.
2. Se envuelve en un `TranslationDecorator`.
3. Se envuelve todo lo anterior en un `SentimentDecorator`.

Cuando llamamos a `generate_summary()`, el texto viaja hacia adentro, se resume, el resultado sale hacia el traductor, se traduce al español, y finalmente el analizador de sentimientos le pega la etiqueta de estrellas (ej. *5 stars*) al final del *string* antes de mostrarlo al usuario.

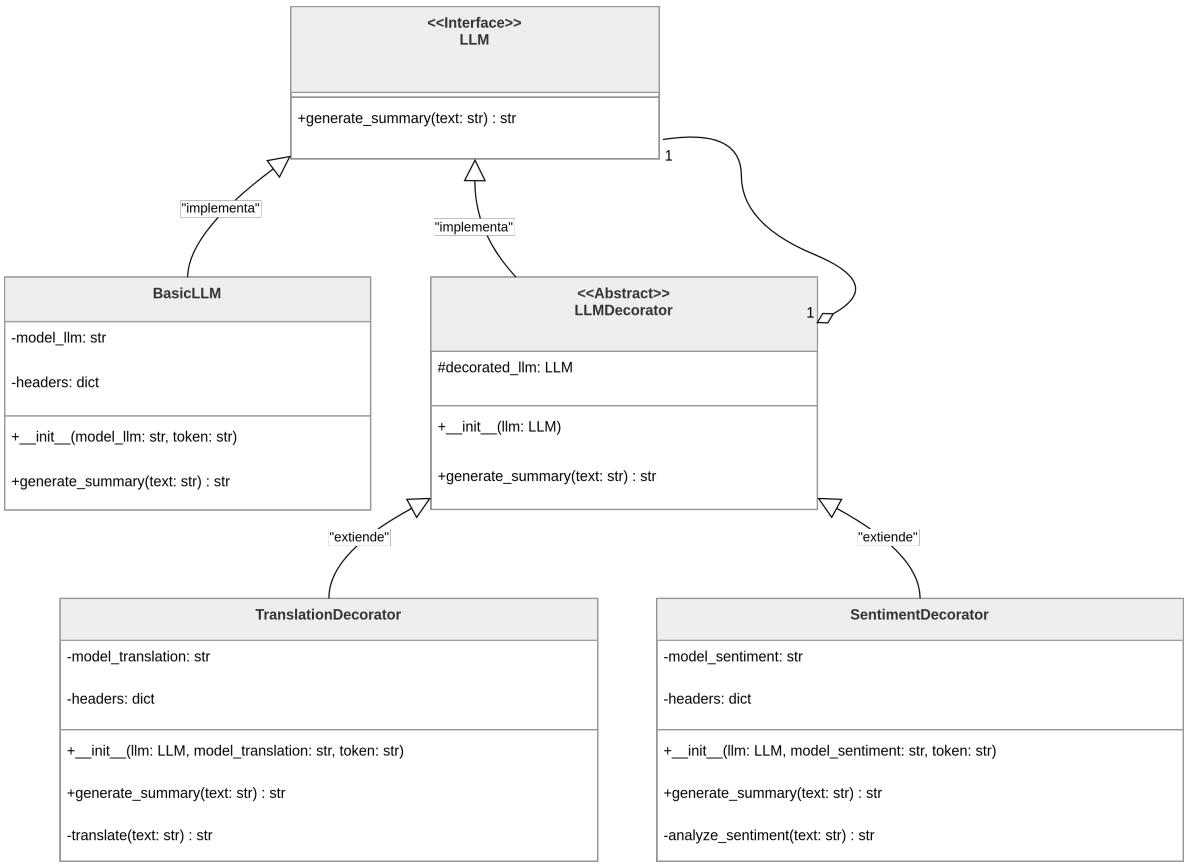
```
llm_completo = SentimentDecorator(TranslationDecorator(llm_base, modelo_trad, token),modelo_sent, token)
resumen_total = llm_completo.generate_summary(texto_original)
print(f"Resultado:\n{resumen_total}\n")
```

2.4. Modo Interactivo y Flexibilidad del Patrón (Extra)

Como mejora de la práctica, hemos implementado un menú en la terminal que permite al usuario construir su propia ÏA a medida". El usuario puede elegir el orden y la cantidad de los envoltorios, demostrando la flexibilidad total del patrón Decorator. Da la posibilidad de generar la serie de pasos que se quiera de forma dinámica y luego mostrar el resultado final por pantalla.

Como el enunciado solicita mostrar primero la salida estándar, el programa imprime por terminal las salidas fijas de las 4 pruebas base y, a continuación, lanza este modo interactivo.

2.5. Diagrama de clases UML



2.6. Pruebas de Ejecución

```
RUEBA 1: Resumen Básico
[BasicLLM] Generando resumen básico...
Resultado:
The V8 engine is a legendary internal combustion engine configuration where eight cylinders are arranged in two banks of four, sharing a single crankshaft. Historically, th
ese engines have been highly revered in the automotive industry and the racing world for their incredible smoothness. For decades, the V8 was the beating heart of American
muscle cars and high-end European sports cars.


PRUEBA 2: Resumen Traducido al Español
[BasicLLM] Generando resumen básico...
[TranslationDecorator] Traduciendo el texto al español...
Resultado:
El motor V8 es una legendaria configuración de motor de combustión interna donde ocho cilindros están dispuestos en dos bancos de cuatro, compartiendo un solo cigüeñal. His
tóricamente, estos motores han sido muy venerados en la industria automotriz y el mundo de las carreras por su increíble suavidad. Durante décadas, el V8 fue el corazón lat
ido de los coches de músculo americano y coches deportivos europeos de alta gama.


PRUEBA 3: Resumen en Inglés + Análisis de Sentimiento
[BasicLLM] Generando resumen básico...
[SentimentDecorator] Analizando el sentimiento...
Resultado:
The V8 engine is a legendary internal combustion engine configuration where eight cylinders are arranged in two banks of four, sharing a single crankshaft. Historically, th
ese engines have been highly revered in the automotive industry and the racing world for their incredible smoothness. For decades, the V8 was the beating heart of American
muscle cars and high-end European sports cars.

[Etiqueta de Sentimiento: 5 stars]


PRUEBA 4: Combinación Total (Traducido y Analizado)
[BasicLLM] Generando resumen básico...
[TranslationDecorator] Traduciendo el texto al español...
[SentimentDecorator] Analizando el sentimiento...
Resultado:
El motor V8 es una legendaria configuración de motor de combustión interna donde ocho cilindros están dispuestos en dos bancos de cuatro, compartiendo un solo cigüeñal. His
tóricamente, estos motores han sido muy venerados en la industria automotriz y el mundo de las carreras por su increíble suavidad. Durante décadas, el V8 fue el corazón lat
ido de los coches de músculo americano y coches deportivos europeos de alta gama.

[Etiqueta de Sentimiento: 5 stars]

=== FIN DEL PROGRAMA ===
```

```
==================================================
MODO INTERACTIVO: Construye tu propia IA
==================================================

¿Quieres crear tu propia cadena de IA personalizada? (s/n): s

Empezando con la IA Base (Resumidor)...

Tu cadena actual es: Resumen Base
¿Qué más quieres añadirle?
1. Añadir Traducción al Español
2. Añadir Análisis de Sentimiento
3. Ejecutar la cadena actual
4. Cancelar
Elige una opción (1-4): 1
Traductor añadido.

Tu cadena actual es: Resumen Base -> Traducción
¿Qué más quieres añadirle?
1. Añadir Traducción al Español
2. Añadir Análisis de Sentimiento
3. Ejecutar la cadena actual
4. Cancelar
Elige una opción (1-4): 2
Sentimiento añadido.

Tu cadena actual es: Resumen Base -> Traducción -> Sentimiento
¿Qué más quieres añadirle?
1. Añadir Traducción al Español
2. Añadir Análisis de Sentimiento
3. Ejecutar la cadena actual
4. Cancelar
Elige una opción (1-4): 2
Sentimiento añadido.

Tu cadena actual es: Resumen Base -> Traducción -> Sentimiento -> Sentimiento
¿Qué más quieres añadirle?
1. Añadir Traducción al Español
2. Añadir Análisis de Sentimiento
3. Ejecutar la cadena actual
4. Cancelar
Elige una opción (1-4): 3

Procesando texto con: Resumen Base -> Traducción -> Sentimiento -> Sentimiento
[BasicLLM] Generando resumen básico...
[TranslationDecorator] Traduciendo el texto al español...
[SentimentDecorator] Analizando el sentimiento...
[SentimentDecorator] Analizando el sentimiento...

========================================
RESULTADO DE TU CADENA:
========================================
El motor V8 es una legendaria configuración de motor de combustión interna donde ocho cilindros están dispuestos en dos bancos de cuatro, compartiendo un solo cigüeñal. His
tóricamente, estos motores han sido muy venerados en la industria automotriz y el mundo de las carreras por su increíble suavidad. Durante décadas, el V8 fue el corazón lat
ido de los coches de músculo americano y coches deportivos europeos de alta gama.
```

3. Ejercicio 3: Extracción de Datos Estadísticos (Web Scraping)

3.1. Arquitectura y Patrones de Diseño Aplicados

Para la resolución de este ejercicio se ha implementado el **Patrón de Diseño Strategy**, como se indica en el enunciado. Este patrón de comportamiento es el adecuado para este escenario, ya que nos permite definir algoritmos de extracción de datos (scraping), encapsular cada uno de ellos y nos permite poder seleccionar el que se quiera en tiempo de ejecución.

- **Clase Abstracta** `EstrategiaScraping`: Define el contrato general para todas las estrategias. Contiene el método abstracto `extraer_datos(urlBase, numPaginas)`, garantizando que cualquier estrategia concreta devuelva siempre una lista con la información estructurada.

```python
from abc import ABC, abstractmethod

class EstrategiaScraping(ABC):

    # Clase abstracta que define el contrato para todas las estrategias de scraping.

    @abstractmethod
    def extraer_datos(self, urlBase: str, numPaginas: int) -> list:
        # Extrae la información de los equipos de hockey.
        # Debe devolver una lista de diccionarios con los datos extraídos.
        pass
```

- **Estrategias Concretas** (`EstrategiaBeautifulSoup` y `EstrategiaSelenium`): Implementan la funcionalidad `extraer_datos`. Cada una encapsula la lógica específica de su herramienta. Mientras que una realiza peticiones HTTP estáticas, la otra instancia un navegador real. Permitiendo elegir la que más nos interese.

- **Clase** `ContextoScraping`: Actúa como intermediario entre el cliente en este caso nuestra clase `main` y las estrategias. Mantiene una referencia a la estrategia actual y delega en ella la tarea de extracción.

```python
from estrategiaScraping import EstrategiaScraping

class ContextoScraping:
    def __init__(self, estrategia: EstrategiaScraping):
        self.estrategia = estrategia

    def ejecutar_estrategia(self, urlBase: str, numPaginas: int):
        return self.estrategia.extraer_datos(urlBase, numPaginas)
```

3.2. Flujo de Ejecución y Extracción de Datos

El programa está diseñado para ser interactivo y robusto frente a fallos en la estructura de la página web:

- **Selección Dinámica**: Al iniciar la ejecución, se solicita al usuario mediante la terminal que elija qué motor de scraping desea utilizar (Opción 1 para BeautifulSoup, Opción 2 para Selenium). El contexto se inicializa en base a esta elección.

```
Seleccione la estrategia de scraping que desea utilizar:

1. BeautifulSoup
2. Selenium

Ingrese el número de la opción deseada (1 o 2):
```

- **Inicialización del contexto**:

```
if opcion == "1":
    contexto = ContextoScraping(EstrategiaBeautifulSoup())
elif opcion == "2":
    contexto = ContextoScraping(EstrategiaSelenium())
else:
    print("Opción no válida.")
    raise SystemExit("Saliendo del programa.\n")
```

- **Navegación y Paginación**: En ambas técnicas la navegación por la página se realiza mediante un bucle `for`, el programa iterará sobre las 5 primeras páginas de la URL base (`https://www.scrapethissite.com/pages/forms/`), modificando dinámicamente el parámetro `?page_num=` en cada iteración.

```
# Construimos la URL de la página actual
url_actual = f"{url_base}?page_num={pagina}"
```

- **Manejo de Excepciones**: Sobre todo en el caso de Selenium hay que tomar precaución por si algún campo que se busca no aparece en la web. De no tomar medidas si se busca un dato en concreto y no está la ejecución se detiene. Para evitar esto se han añadido dos bloques `try` para que en caso de tener algún error relacionado con esto se ponga 0 como dato y no suceda nada impredecible.

```
try:
    ot_losses = equipo.find_element(By.CLASS_NAME, "ot-losses").text.strip()
except:
    ot_losses = "0"

try:
    win_pct = equipo.find_element(By.CLASS_NAME, "pct").text.strip()
except:
    win_pct = "0"
```

3.3. Implementación de las Estrategias (BeautifulSoup vs. Selenium)

Para cumplir con los requisitos obligatorios, se han desarrollado dos enfoques técnicos distintos:

- **Aproximación con BeautifulSoup**: Utiliza la librería `requests` para descargar el HTML estático, este nos devuelve un texto a lo grande, buscar información ahí sería muy complicado. Para solucionarlo usamos la librería `bs4` que parsea el texto, permitiendo realizar búsquedas de etiquetas `<tr>` con gran velocidad y bajo coste computacional.

```
try:
    ot_losses = equipo.find_element(By.CLASS_NAME, "ot-losses").text.strip()
except:
    ot_losses = "0"

try:
    win_pct = equipo.find_element(By.CLASS_NAME, "pct").text.strip()
except:
    win_pct = "0"
```

- **Aproximación con Selenium**: Utiliza `webdriver` para instanciar un navegador Chrome real en modo `headless` (sin interfaz gráfica, utilizando `Options`), hemos comentado esa línea para que se pueda ver cómo se abre el navegador; de querer ocultarlo basta con descomentar. Esta estrategia procesa el HTML renderizado, buscando elementos mediante `By.CLASS_NAME`. Se ha incluido un bloque `try...finally` para asegurar la ejecución del comando `driver.quit()` y liberar los recursos del sistema una vez finalizada la extracción o en caso de error crítico.

```
driver = webdriver.Chrome(options=opciones)
```

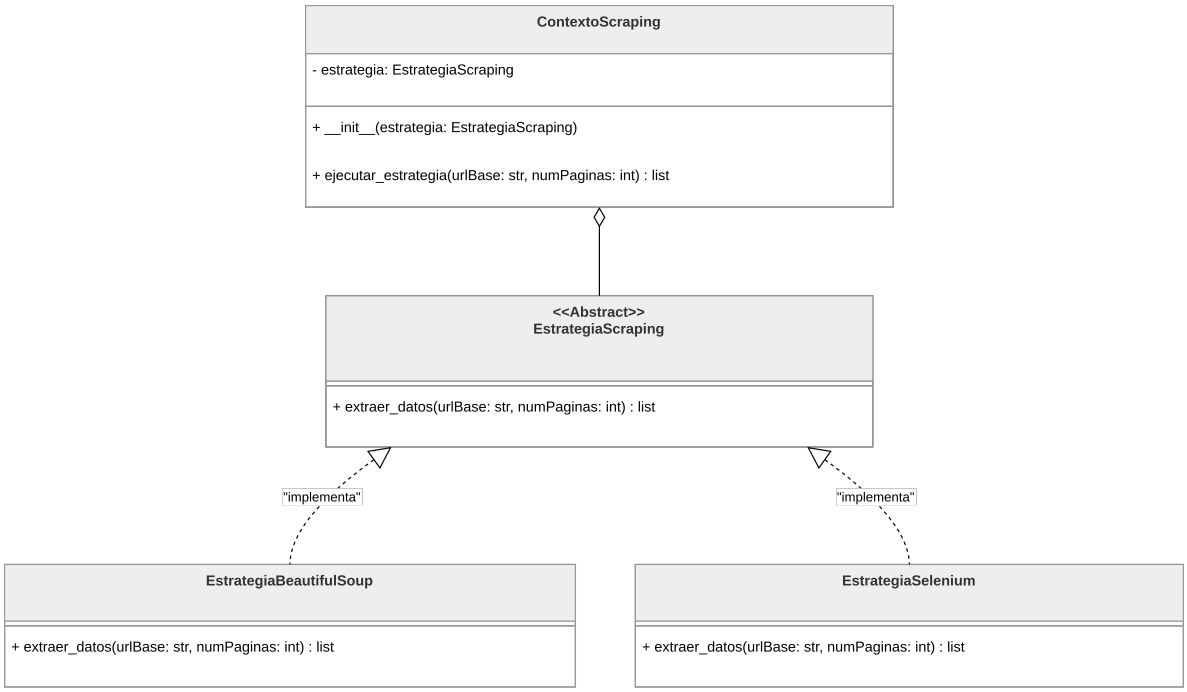
```
equipos = driver.find_elements(By.CLASS_NAME, "team")
```

3.4. Almacenamiento

Una vez que la estrategia seleccionada devuelve la lista de diccionarios con los 9 campos requeridos (Team Name, Year, Wins, Losses, OT Losses, Win %, Goals For, Goals Against, +/-), se debe de escribir en un archivo que se genere. Por lo que, se ha creado una función `guardar_datos_csv` que hace uso del módulo estándar `csv` de Python. A través de la clase `DictWriter`, se extraen dinámicamente las claves del primer diccionario para generar la fila de cabeceras (`writeheader`) y, posteriormente, se vuelcan todos los registros mediante `writerows()`, generando el archivo final `estadisticas_hockey.csv`.

```
equipos = driver.find_elements(By.CLASS_NAME, "team")
```

3.5. Diagrama de clases UML



3.6. Pruebas de Ejecución

```
Ingrese el número de la opción deseada (1 o 2): 1
Iniciando el proceso de scraping...

Iniciando scraping con BeautifulSoup para 5 páginas...
Scrapeando página 1...
Scrapeando página 2...
Scrapeando página 3...
Scrapeando página 4...
Scrapeando página 5...
Guardando los datos en estadisticas_hockey.csv...
¡Archivo guardado con éxito!
Programa finalizado
```

DS_Practicas > P1 > Ejercicio3 > estadisticas_hockey.csv

```csv
Team Name,Year,Wins,Losses,OT Losses,Win %,Goals For,Goals Against,+ / -
Boston Bruins,1990,44,24,,0.55,299,264,35
Buffalo Sabres,1990,31,30,,0.388,292,278,14
Calgary Flames,1990,46,26,,0.575,344,263,81
Chicago Blackhawks,1990,49,23,,0.613,284,211,73
Detroit Red Wings,1990,34,38,,0.425,273,298,-25
Edmonton Oilers,1990,37,37,,0.463,272,272,0
Hartford Whalers,1990,31,38,,0.388,238,276,-38
Los Angeles Kings,1990,46,24,,0.575,340,254,86
Minnesota North Stars,1990,27,39,,0.338,256,266,-10
Montreal Canadiens,1990,39,30,,0.487,273,249,24
New Jersey Devils,1990,32,33,,0.4,272,264,8
New York Islanders,1990,25,45,,0.312,223,290,-67
New York Rangers,1990,36,31,,0.45,297,265,32
Philadelphia Flyers,1990,33,37,,0.412,252,267,-15
```

4. Ejercicio 4: Patrón Intercepting Filter

4.1. Introducción

En esta práctica se implementa un sistema de validación de credenciales utilizando el patrón de diseño **Intercepting Filter**. El sistema valida un correo electrónico y una contraseña mediante una serie de filtros independientes antes de realizar la autenticación.

4.2. Descripción de la solución

Se han implementado distintos filtros para validar:

- El formato del correo (texto antes del @).
- El dominio del correo (gmail.com o hotmail.com).
- La seguridad de la contraseña (longitud mínima, número y carácter especial).

Cada validación se realiza en una clase independiente, permitiendo una arquitectura modular y fácilmente extensible.

4.3. Uso de herencia vs módulos

Inicialmente, los filtros se implementaron utilizando herencia a partir de una clase base `Filter`. Sin embargo, dado que dicha clase únicamente define un método sin implementación, conceptualmente actúa como una interfaz.

Por este motivo, se ha sustituido el uso de herencia por módulos de Ruby mediante `include`, simulando así una interfaz. Esto permite una mayor flexibilidad en el diseño y se adapta mejor al modelo de Ruby.

4.4. Diagrama UML

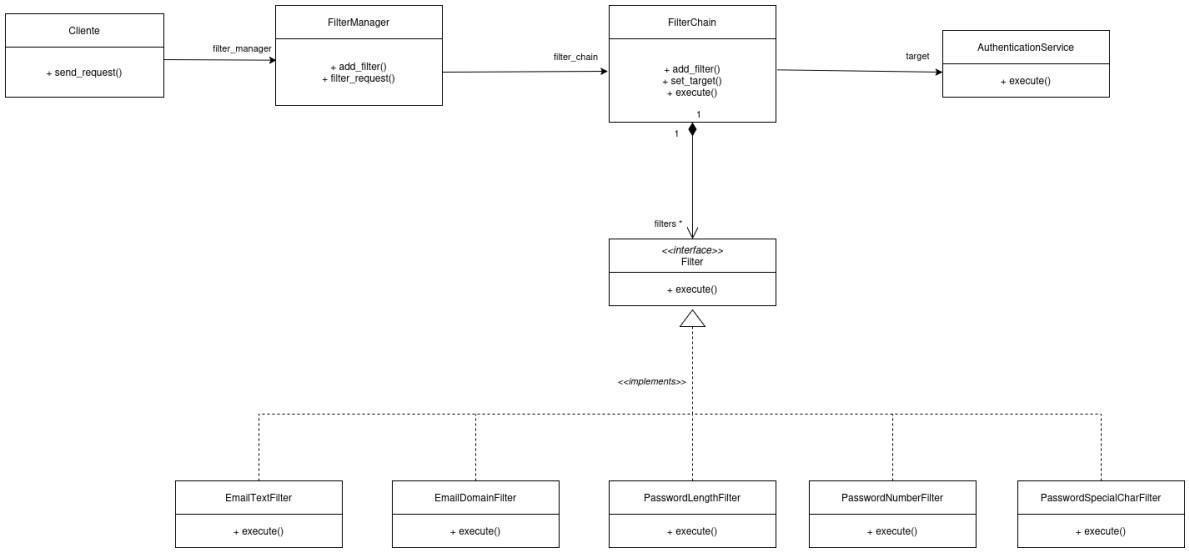


Figura 1: Diagrama UML del sistema

El diagrama representa la estructura del patrón Intercepting Filter, incluyendo la relación de composición entre la cadena de filtros y los filtros, así como la implementación de la interfaz mediante módulos.

4.5. Ejemplos de ejecución

4.5.1. Caso válido

```
usuario@~/DS/practicas/P1_ej4$ ruby main.rb
Introduce tu correo: a@gmail.com
Introduce tu contraseña: hola123!
Autenticación correcta para a@gmail.com
```

Figura 2: Caso de uso válido

4.5.2. Caso inválido: Formato incorrecto

```
usuario@~/DS/practicas/P1_ej4$ ruby main.rb
Introduce tu correo: @gmail.com
Introduce tu contraseña: hola123!
Error de validación: El correo debe tener texto antes del @
```

Figura 3: Formato incorrecto

4.5.3. Caso inválido: Dominio incorrecto

```
usuario@~/DS/practicas/P1_ej4$ ruby main.rb
Introduce tu correo: agmail.com
Introduce tu contraseña: hola123!
Error de validación: Dominio no permitido
```

Figura 4: Dominio incorrecto

4.5.4. Caso inválido: Longitud de contraseña incorrecta

```
usuario@~/DS/practicas/P1_ej4$ ruby main.rb
Introduce tu correo: a@hotmail.com
Introduce tu contraseña: h123!
Error de validación: Contraseña demasiado corta
```

Figura 5: Longitud de contraseña incorrecta

4.5.5. Caso inválido: Contraseña con al menos un número

```
usuario@~/DS/practicas/P1_ej4$ ruby main.rb
Introduce tu correo: a@hotmail.com
Introduce tu contraseña: hola!!!!!!!
Error de validación: Debe contener un número
```

Figura 6: Contraseña sin número

4.5.6. Caso inválido: Contraseña con al menos un carácter especial

```
usuario@~/DS/practicas/P1_ej4$ ruby main.rb
Introduce tu correo: a@gmail.com
Introduce tu contraseña: hola123123123
Error de validación: Debe contener un carácter especial
```

Figura 7: Contraseña sin carácter especial

4.6. Conclusiones

El uso del patrón Intercepting Filter permite implementar validaciones de forma modular y desacoplada. Cada filtro se encarga de una única responsabilidad, facilitando la ampliación del sistema.

El uso de módulos en lugar de herencia refuerza el diseño al tratar el filtro como una interfaz, adaptándose mejor a las características del lenguaje Ruby.